TÍTULO DO PROJETO: Sistema Embarcado de Estação Meteorológica IoT com ESP32
ALUNOS: Roberto O. de Andrade Junior e Gabriel Raul Marino
DATA: 03/12/2025

1. Introdução e Objetivos

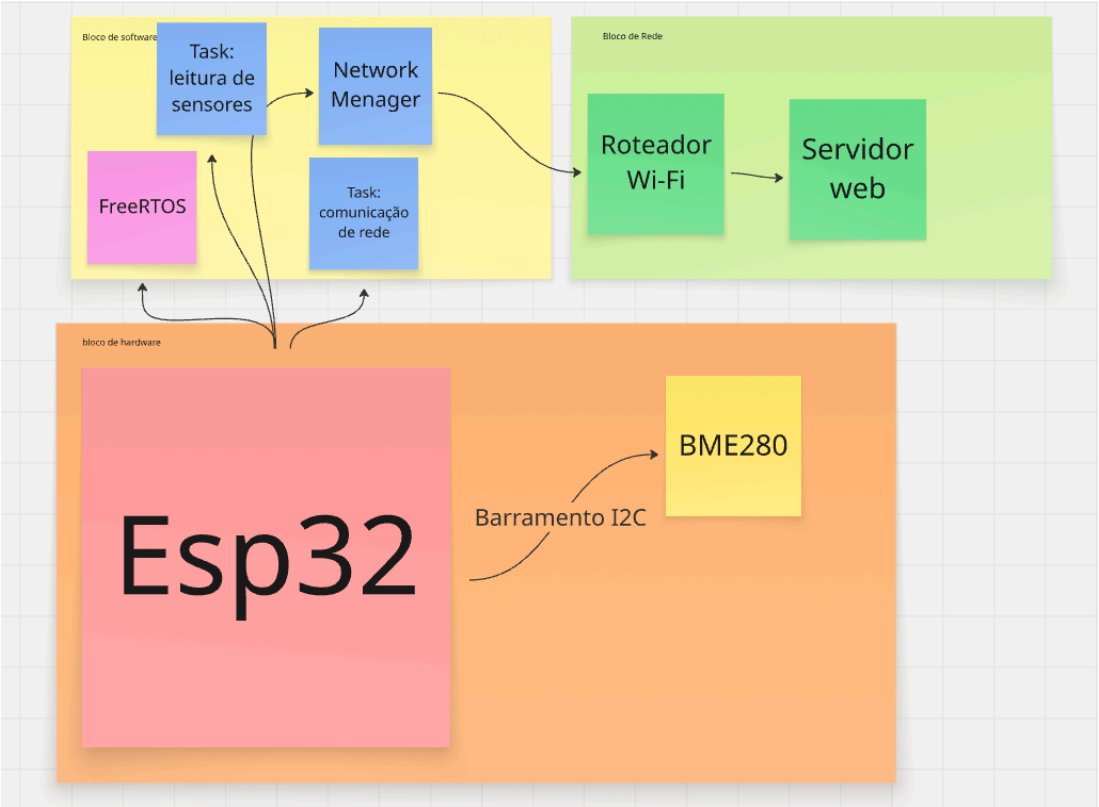
O objetivo deste projeto foi desenvolver um sistema embarcado robusto para monitoramento ambiental (Estação Meteorológica), utilizando técnicas avançadas de programação em C++ moderno e arquitetura orientada a objetos. O sistema é capaz de realizar a leitura de temperatura, umidade e pressão atmosférica, disponibilizando esses dados em tempo real através de uma interface Web hospedada no próprio dispositivo.

O hardware base utilizado foi o microcontrolador **ESP32** (devido ao seu suporte nativo a Wi-Fi e FreeRTOS) e o sensor ambiental **BME280**, que se comunica via barramento I2C. O foco principal do desenvolvimento foi a estruturação do software, garantindo desacoplamento entre drivers e aplicação, uso de templates, tratamento de exceções e persistência de dados.

2. Arquitetura do Sistema e Decisões de Projeto

A arquitetura foi projetada em camadas para cumprir o requisito de separação entre a Camada de Aplicação e a HAL (Hardware Abstraction Layer).

Durante a fase inicial de concepção, foi avaliada a utilização de plataformas de IoT em nuvem de terceiros (como o Blynk) para a interface do usuário. Entretanto, a equipe optou por pivotar a solução para um **Servidor Web Local embarcado** (Web Server).



Esta decisão estratégica foi tomada visando dois objetivos principais:

1. **Independência de Serviços Externos:** Garantir que o sistema funcione em redes locais (Intranet) sem necessidade de acesso constante à internet ou dependência de servidores de terceiros.
2. **Demonstração de Competência Técnica:** A implementação de um servidor HTTP nativo permitiu um controle mais granular sobre as requisições (GET/POST) e o uso aprofundado das APIs de rede do ESP-IDF em C++, cumprindo com maior rigor os requisitos de implementação de software do projeto.

2.1. Camada de Abstração de Hardware (HAL)

Para evitar o acoplamento direto com a API específica do ESP-IDF, foi desenvolvida a classe `I2CDriver`. Esta classe encapsula as funções de baixo nível (`i2c_master_write`, `i2c_driver_install`), expondo métodos amigáveis como `writeReg` e `readReg`. Isso permite que a lógica dos sensores permaneça inalterada mesmo se o microcontrolador subjacente for trocado.

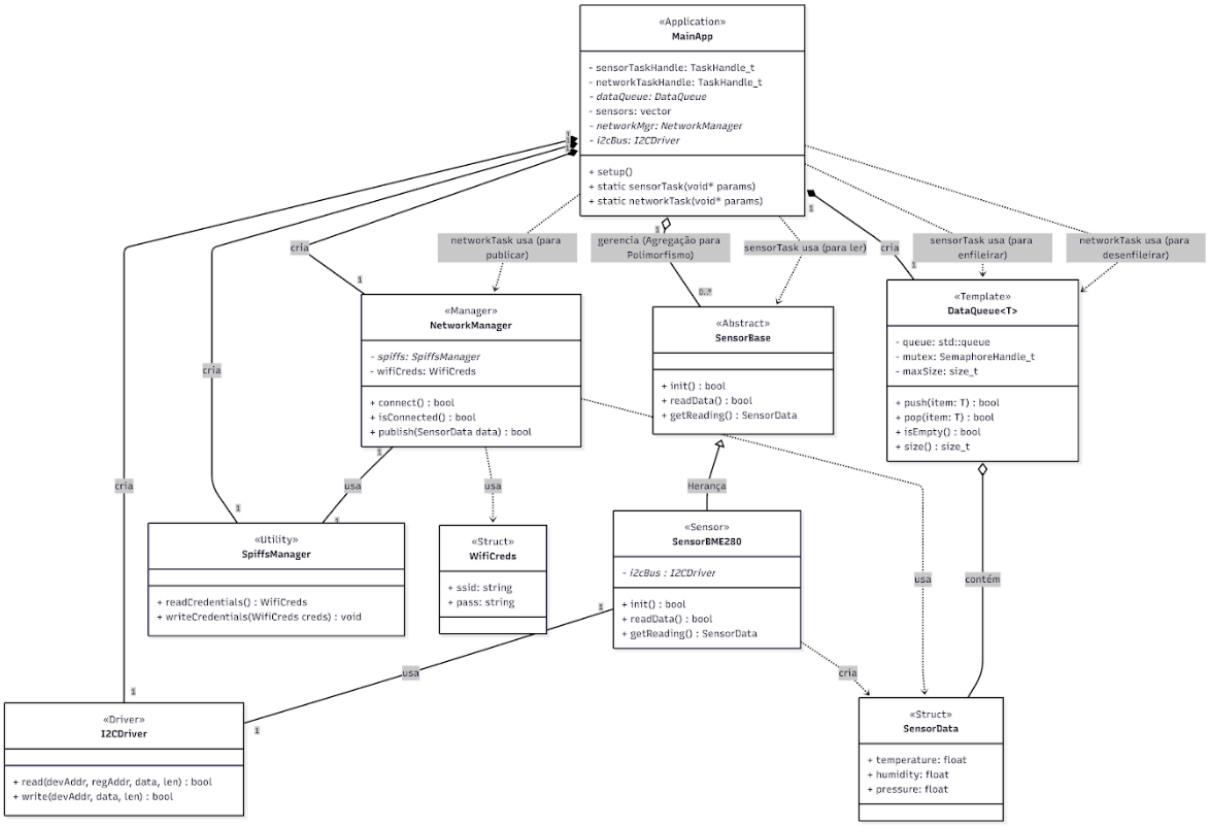
2.2. Sistema Operacional e Agendamento (RTOS)

O sistema utiliza o **FreeRTOS** para o agendamento de tarefas, garantindo um comportamento não-bloqueante. Foram definidas as seguintes estratégias:

- **Tarefa de Leitura (`sensorReadTask`):** Executa periodicamente a coleta de dados dos sensores. O uso de `vTaskDelay` permite que a CPU entre em estado de baixo consumo ou processe outras tarefas (como o Wi-Fi) nos intervalos.
- **Thread-Safety:** O acesso aos dados compartilhados entre a tarefa de leitura e o servidor web é protegido por **Mutexes**, implementados dentro da classe template `DataQueue`, evitando condições de corrida (race conditions).

3. Modelagem e Orientação a Objetos

O projeto demonstra o uso extensivo dos pilares da Orientação a Objetos.



3.1. Herança e Classes Abstratas

Foi definida uma classe base abstrata `SensorBase` (arquivo `SensorBase.h`), que estabelece um contrato para todos os sensores através de métodos virtuais puros:

virtual bool init() = 0;
virtual bool readData() = 0;
virtual SensorData getReading() = 0;

A classe concreta `BME280Sensor` herda de `SensorBase` e implementa a comunicação específica com o hardware da Bosch.

3.2. Polimorfismo

O controlador principal (`DeviceController`) gerencia os sensores de forma polimórfica utilizando um vetor de ponteiros para a classe base (`std::vector<SensorBase*> sensors`). Isso permite que o sistema itere sobre uma lista de sensores genéricos para inicializá-los e lê-los, sem precisar conhecer o tipo específico de cada um em tempo de compilação.

3.3. Encapsulamento

A complexidade dos cálculos de compensação do sensor BME280 (que envolvem leitura de registradores de calibração e fórmulas bit-a-bit) foi totalmente encapsulada como métodos

`private` dentro da classe `BME280Sensor`. O usuário da classe interage apenas com métodos públicos simples, garantindo a integridade dos dados internos.

4. Recursos Avançados de C++

4.1. Templates e Estruturas Genéricas

Foi desenvolvida a classe `DataQueue<T>` (arquivo `DataQueue.h`) para atuar como um buffer seguro entre tarefas. O uso de **Templates** permite que essa estrutura seja reutilizada para qualquer tipo de dado, não apenas a struct `SensorData`, demonstrando flexibilidade e reuso de código.

4.2. Tratamento de Arquivos e Persistência (SPIFFS)

Para cumprir o requisito de interação com arquivos, o sistema implementa a classe `SpiffsManager`, que gerencia uma partição dedicada na memória Flash do ESP32 usando o sistema de arquivos SPIFFS.

A principal aplicação deste recurso é na **Camada de Apresentação (Interface Web)**:

- **Leitura Dinâmica:** O arquivo da interface gráfica (`index.html`) é armazenado independentemente do firmware. Quando um usuário acessa o IP do dispositivo, a classe `NetworkManager` abre este arquivo, lê seu conteúdo em *chunks* (pedaços) e envia ao navegador.
- **Vantagem Técnica:** Essa abordagem economiza memória RAM crítica (Heap), pois não exige que o código HTML inteiro fique carregado em variáveis globais. Além disso, permite a atualização do design da página web (Front-end) sem a necessidade de recompilar e regravar todo o código C++ (Back-end).

5. Testes e Resultados

5.1. Testes de Unidade

Foi validada a classe `DataQueue<T>` garantindo que o Mutex (semáforo binário) bloqueie corretamente o acesso simultâneo. Testes de borda confirmaram que a fila reporta corretamente estados de "vazia" ou "cheia" sem causar travamento do sistema (crash).

5.2. Testes de Integração e Sistema

O sistema foi submetido a testes práticos de funcionamento:

1. **Inicialização:** O log do sistema confirmou a detecção correta do sensor BME280 no endereço `0x76` e a montagem do sistema de arquivos SPIFFS.
2. **Falhas de Hardware:** Ao desconectar o sensor propositalmente, o método `init()` retornou `false` e o sistema tratou a exceção logando o erro, mantendo o servidor

web ativo (degradação graciosa)

```
I (532) MAIN: === Iniciando Estacao Meteorologica IoT ===
I (532) SPIFFS: Inicializando SPIFFS...
I (542) SPIFFS: SPIFFS montado com sucesso!
I (542) DeviceController: Iniciando DeviceController...
I (542) BME280: Chip ID lido: 0x0 (Esperado: 0x60)
E (542) BME280: Erro: BME280 não encontrado!
E (552) DeviceController: FALHA: Sensor BME280 não encontrado!
```

```
I (772) wifi:enable tsf
I (772) WebManager: Conectando ao WiFi...
I (5782) WebManager: Iniciando Web Server...
I (5782) WebManager: Servidor Iniciado! Verifique o IP no monitor serial.
I (5782) MAIN: Sistema iniciado. app_main entrando em loop inativo.
```

3. **Monitoramento:** A interface web exibiu os dados de temperatura, umidade e pressão atualizados a cada 2 segundos, confirmando o funcionamento da cadeia completa (Sensor -> I2C -> Controlador -> Queue -> WebServer).

```
I (713) wifi:config NVS flash: enabled
I (713) wifi:config nano formatting: disabled
I (713) wifi:Init data frame dynamic rx buffer num: 32
I (723) wifi:Init static rx mgmt buffer num: 5
I (723) wifi:Init management short buffer num: 32
I (723) wifi:Init dynamic tx buffer num: 32
I (733) wifi:Init static rx buffer size: 1600
I (733) wifi:Init static rx buffer num: 10
I (743) wifi:Init dynamic rx buffer num: 32
I (743) wifi_init: rx ba win: 6
I (743) wifi_init: accept mbox: 6
I (753) wifi_init: tcpip mbox: 32
I (753) wifi_init: udp mbox: 6
I (753) wifi_init: tcp mbox: 6
I (753) wifi_init: tcp tx win: 5760
I (763) wifi_init: tcp rx win: 5760
I (763) wifi_init: tcp mss: 1440
I (763) wifi_init: WiFi IRAM OP enabled
I (773) wifi_init: WiFi RX IRAM OP enabled
W (773) wifi:Password length matches WPA2 standards, authmode threshold changes from OPEN to WPA2
I (783) phy_init: phy_version 4861,b71b5ad,Aug  5 2025,11:16:06
I (863) wifi:mode : sta (ec:94:cb:6a:e8:00)
I (863) wifi:enable tsf
I (863) WebManager: Conectando ao WiFi...
I (883) wifi:new:<11,0>, old:<1,0>, ap:<255,255>, sta:<11,0>, prof:1, snd_ch_cfg:0x0
I (883) wifi:state: init -> auth (0xb0)
I (883) wifi:state: auth -> assoc (0x0)
I (893) wifi:state: assoc -> run (0x10)
I (903) wifi:<ba-add>idx:0 (ifx:0, cc:58:30:03:a0:73), tid:5, ssn:0, winSize:64
I (903) wifi:<ba-add>idx:1 (ifx:0, cc:58:30:03:a0:73), tid:0, ssn:0, winSize:64
I (923) wifi:connected with Roberto_2G, aid = 4, channel 11, BW20, bssid = cc:58:30:03:a0:73
I (923) wifi:security: WPA2-PSK, phy: bgn, rssi: -69
I (933) wifi:pm start, type: 1

I (933) wifi:dp: 1, bi: 102400, li: 3, scale listen interval from 307200 us to 307200 us
I (1173) wifi:AP's beacon interval = 102400 us, DTIM period = 1
I (1993) esp_netif_handlers: sta ip: 192.168.0.16, mask: 255.255.255.0, gw: 192.168.0.1
I (5873) WebManager: Iniciando Web Server...
I (5873) WebManager: Servidor Iniciado! Verifique o IP no monitor serial.
I (5873) MAIN: Sistema iniciado. app_main entrando em loop inativo.
I (5873) DeviceController: Leitura -> Temp: 26.680 C | Hum: 71.400 % | Pres: 1008.988 hPa
I (7883) DeviceController: Leitura -> Temp: 26.690 C | Hum: 71.356 % | Pres: 1008.980 hPa
```



6. Conclusão

O projeto atingiu todos os objetivos propostos, entregando um sistema funcional que integra hardware e software de forma modular. O uso de C++ moderno permitiu criar uma arquitetura limpa, onde a adição de novos sensores requer apenas a criação de uma nova classe derivada de `SensorBase`, sem necessidade de alterar a lógica de controle ou de rede. O sistema demonstrou estabilidade graças ao uso correto do FreeRTOS e mecanismos de proteção de memória.